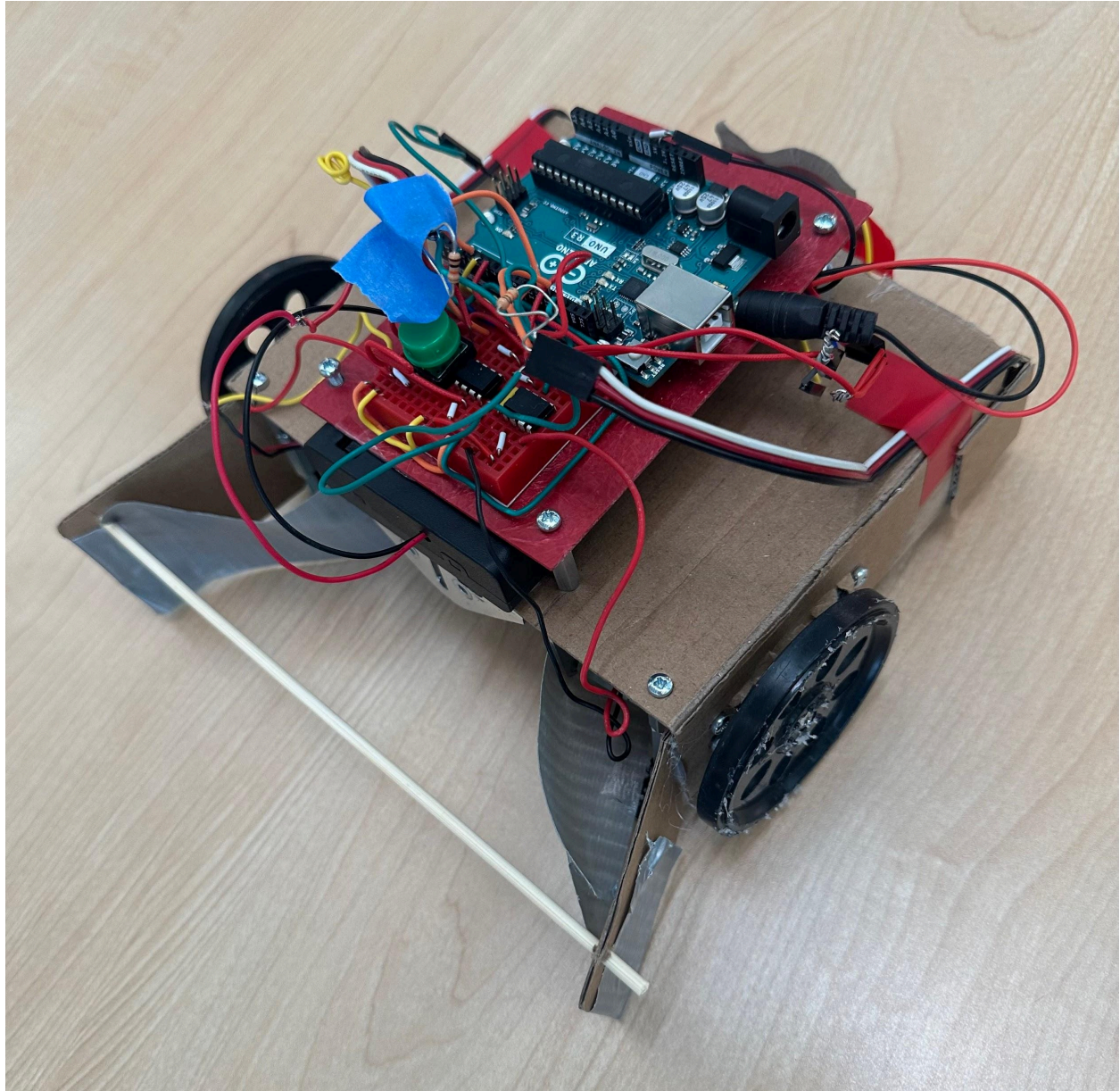


Robot Project Final Report

Authors: Sara Pedraja (sp2297), Eduardo Antonio Juarez (ea398), Mia Paz (mkp86)



Design and Strategy Overview

The original strategy for the robot was to drive toward the center of the board and, upon detecting the opposing color, turn and drive into nearby blocks to collect them. The robot was designed with a one-way flap mechanism intended to prevent collected blocks from falling out when reversing. In addition, the design

incorporated an ultrasonic sensor to detect cubes outside the robot's immediate path and redirect the robot toward them.

The robot's mechanical structure consisted of a cardboard enclosure with extended front arms designed to funnel cubes into the collection area. The electrical system included QTI sensors for edge detection, a color sensor for board navigation, and planned integration of an ultrasonic sensor for object detection. The software controlled navigation and sensor-based decision making during competition.

A layout and diagram of the robot's original intended design are provided in Appendix C.

Design Process Reflection

Our group worked through the design process by building, testing, and making changes as we went. Our original plan for our robot to collect cubes included using a one-way door using hinges, an ultrasonic sensor to help detect cubes, and larger wheels to give the robot more clearance.

However, as we started building and testing, we ran into several problems that forced us to change our design. The plastic hinges we ordered to use for the door were too stiff, making it impractical for the cubes to get through. Thus, we decided to focus on attaching cardboard arms at the front of the robot to act like a funnel. We also programmed the robot to only move forward so cubes wouldn't fall back out.

We also tried to use an ultrasonic sensor to help the robot find cubes, but it didn't work properly when connected with the other sensors because it wasn't getting enough power. Since we couldn't fix that given our limited time and resources, we decided to remove it and keep the design simpler.

Another issue came up with the wheels. We originally wanted to use larger wheels to raise the robot, but they didn't fit our motors. We tried to make them work by attaching them to the original wheels, but that made the robot too wide and violated the size limit. Since we didn't have much time left, we stuck with the original wheels and instead improved the intake by shortening the back of the robot's cardboard frame and making the front arms longer.

There was also an issue with the QTI sensor cable dragging on the ground, which stopped the wheel from turning properly. To fix this, we adjusted the height of the sensor so it wouldn't drag, though this reduced sensor accuracy.

Overall, we had to change our design multiple times, but each issue helped us improve the robot. By testing often and making adjustments, we were able to complete the milestones and end up with a working design.

Competition Analysis

The robot did not perform as intended during competition day. Our navigation logic relied on the QTI sensors checking for boundary conditions before driving forward. This system functioned reliably during testing both in the lab and at home; however, conditions in the atrium significantly affected sensor performance. The QTI sensors triggered continuously, particularly on the blue sections of the board, causing the robot to repeatedly back up, rotate in place, or drive off the board. To address this issue during the competition, we temporarily taped a white sheet of paper beneath the robot and relied primarily on the color sensor for navigation. This adjustment allowed the robot to operate more consistently and resulted in ties with several other teams.

Another limitation was the robot's storage capacity. Compared to other teams' robots, our enclosure provided less internal space for cube collection, which reduced the robot's overall scoring potential during matches.

Despite these issues, the robot held up mechanically and avoided sustaining any damage during the rounds, which allowed our team to keep competing.

Conclusion

If we were to redo this project, we would improve both the mechanical design and our testing approach. We would consider implementing a gear train to increase speed, use larger wheels that better fit within the dimension constraints, and replace the cardboard frame with a sturdier 3D-printed or laser-cut structure for improved cube collection and stability. We would also complete more of the mechanical assembly earlier to allow additional time for code testing and debugging, since many of our software changes were made last minute and our QTI sensors unexpectedly failed after the first round. Our advice to future students would be to order and fabricate parts as early as possible and avoid overly relying on sensors by designing fallback behaviors that still allow the robot to function if sensors fail.

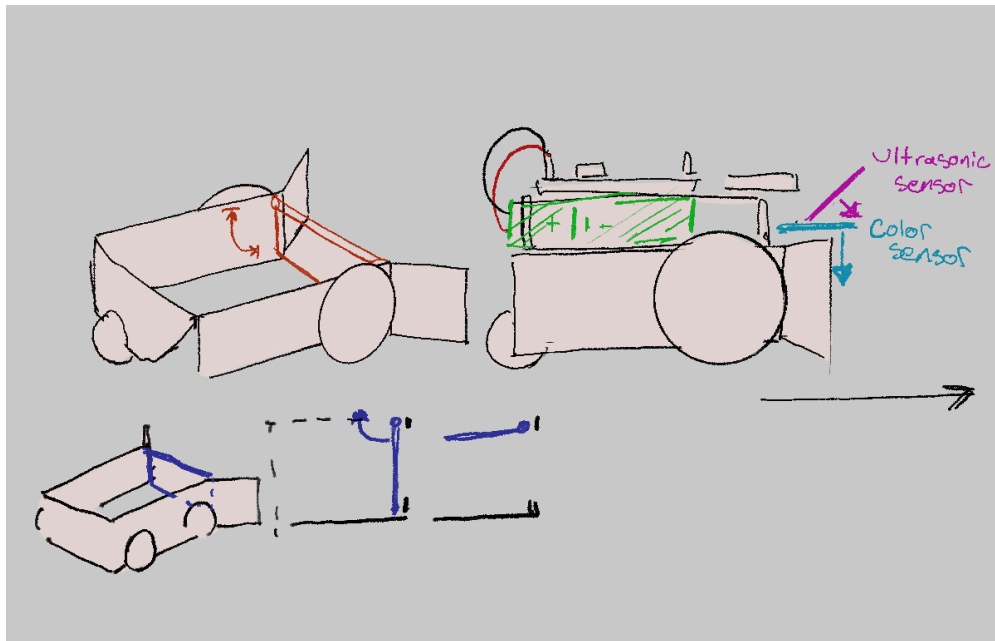
While not entirely successful in competition, we are proud of how hard we worked on this project, and how we were able to come together to produce a final result and overcome each challenge.

Appendix A

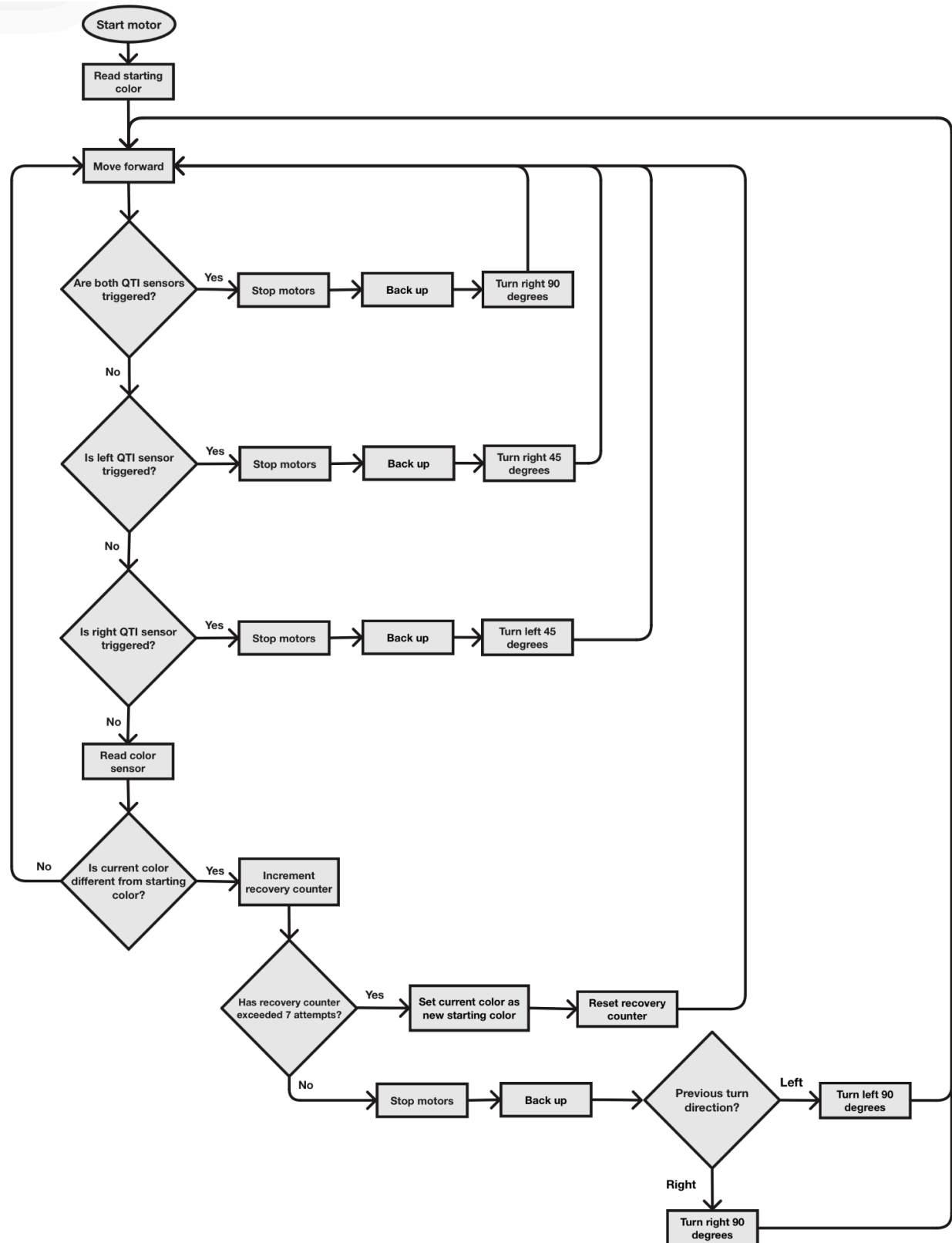
Part Name	Vendor	Item Number	Price	Quantity
Plastic Hinges without Holes	McMaster-Carr	11185A141	\$1.02	2
Wheels	McMaster-Carr	2781T91	\$2.73	2
Corrugated Cardboard Sheet	McMaster-Carr	20585T22	\$11.14	1
			Total	
			\$14.89	



Appendix C



Appendix D



Appendix E

```
// CONSTANTS

// Time delay used to approximate a 90 degree turn.
// This value was tuned for our robot.
#define TIME_TURN_90 330

// Color sensor values for detecting blue.
// These were kept from earlier testing, but the final logic compares
// the current color to the starting color instead of directly using blue.
#define BLUE_MIN 350
#define BLUE_MAX 450

// QTI sensor pins
// The left QTI sensor is connected to Arduino pin 13/PB5.
// The right QTI sensor is connected to Arduino pin 12/PB4.
#define QTI_LEFT_PIN PB5
#define QTI_RIGHT_PIN PB4

// FUNCTION DECLARATIONS

// Initializes motor pins, PWM timers, QTI sensors, color sensor interrupt,
// and Timer1 for color sensor pulse measurement.
void setupMotors(void);

// Movement functions
void forward(void);
void backward(void);
void turnRight(void);
void turnLeft(void);
void stopMotors(void);

// Color sensor functions
int getColor(void);
int isBlue(int val);
```

```
int getColorStart(void);

// QTI sensor functions
int readLeftQTI(void);
int readRightQTI(void);

// GLOBAL VARIABLES

// store the color sensor pulse width
volatile uint16_t timer = 0;

// Stores the measured color sensor period
volatile uint16_t period = 0;

// Counts how many times the robot detects a color different from the starting
color
// This helps prevent the robot from getting stuck trying to return to a color
it cannot find
int otherColorCount = 0;

// Used to average multiple color readings
int sum = 0;

// Stores the initial floor color that the robot starts on
int colorStart = 0;

// Allowed difference between current color and starting color.
// If the difference is larger than this, the robot treats it as a color
change.
int tolerance = 80;

// Tracks the most recent turn direction
// 1 means the last turn was right
// -1 means the last turn was left
int lastTurn = 1;
```

```
// COLOR SENSOR INTERRUPT

// This interrupt runs whenever the color sensor signal on PB0 changes.
// On the rising edge, Timer1 is reset.
// On the falling edge, the current timer value is saved.
// This measures the pulse width from the color sensor.
ISR(PCINT0_vect)
{
    if (PINB & 0b00000001) {
        TCNT1 = 0;           // Reset Timer1 when PB0 goes high
    } else {
        timer = TCNT1;       // Save Timer1 value when PB0 goes low
    }
}

// MAIN PROGRAM

int main(void)
{
    // Set up all motors, sensors, timers, and interrupts.
    setupMotors();

    // Short delay to allow the robot hardware to stabilize after setup
    _delay_ms(200);

    // Start serial communication for debugging sensor values
    Serial.begin(9600);

    // Take 10 color readings and average them to define the starting floor
    color
    colorStart = getColorStart();

    // Print the starting color value for debugging
```

```
Serial.println(colorStart);

_delay_ms(50);

// This variable was originally used for testing decreasing turn times.
int turnTime = 330;

// Main robot control loop
// The robot repeatedly checks its sensors and chooses its movement
response
while(1)
{
    // QTI BORDER OVERRIDE
    // The QTI sensors are checked first.
    // If both QTI sensors detect border, robot backs up and turns right to
move away from edge
    if (readLeftQTI() && readRightQTI())
    {
        stopMotors();

        backward();
        _delay_ms(150);
        stopMotors();

        turnRight();
        _delay_ms(TIME_TURN_90);
        stopMotors();
    }

    // If only left QTI sensor detects border, robot backs up & turns right
slightly to avoid border
    else if (readLeftQTI())
    {
        stopMotors();

        backward();
```

```
        _delay_ms(80);
        stopMotors();

        turnRight();
        _delay_ms(TIME_TURN_90 / 2);
        stopMotors();
    }

    // If only right QTI sensor detects border, robot backs up & turns left
    slightly to avoid border
    else if (readRightQTI())
    {
        stopMotors();

        backward();
        _delay_ms(80);
        stopMotors();

        turnLeft();
        _delay_ms(TIME_TURN_90 / 2);
        stopMotors();
    }

    // NORMAL COLOR-BASED BEHAVIOR
    // If no border is detected, the robot moves forward and checks floor
    color.
    else
    {
        forward();

        // Read the current color sensor value.
        int color = getColor();

        // Print color reading for debugging.
        Serial.println(color);
    }
}
```

```

        // If robot keeps detecting different color & can't return to
original color, it updates the baseline
        // This prevents robot from getting stuck in an endless loop
        if (otherColorCount > 6)
        {
            colorStart = getColorStart(); // Set current surface as new
baseline
            otherColorCount = 0;           // Reset recovery attempt
counter
            turnTime = 330;                // Reset old test variable
        }

        // Compare the current color to the starting color
        // If the difference is greater than the tolerance, robot assumes
        // it has moved onto a different color.
        if (abs(color - colorStart) > tolerance)
        {
            // Count this as one failed attempt to stay on the starting
color.
            otherColorCount += 1;

            // If the previous turn was left, continue turning left during
recovery
            if (lastTurn < 0)
            {
                stopMotors();
                _delay_ms(50);

                backward();
                _delay_ms(120);
                stopMotors();

                turnLeft();
                _delay_ms(TIME_TURN_90);
                stopMotors();
            }
        }
    }
}

```

```
        _delay_ms(100);

        // This was used in testing but does not affect the final
turn delay
        turnTime -= 30;
    }

    // If the previous turn was right, continue turning right
during recovery.
    else
    {
        stopMotors();
        _delay_ms(50);

        backward();
        _delay_ms(120);
        stopMotors();

        turnRight();
        _delay_ms(TIME_TURN_90);
        stopMotors();

        _delay_ms(100);

        // This was used in testing but does not affect the final
turn delay
        turnTime -= 30;
    }
}

// Small delay to prevent the loop from running too rapidly.
_delay_ms(5);
}
}
```

```
// ----- SETUP FUNCTION -----

void setupMotors(void)
{
    // Set motor control pins as outputs.
    // PB3 is used as one motor PWM output.
    DDRB |= 0b00001000;

    // PD3, PD5, and PD6 are used as motor PWM outputs.
    DDRD |= 0b01101000;

    // Set QTI sensor pins as inputs.
    DDRB &= ~(1 << QTI_LEFT_PIN);
    DDRB &= ~(1 << QTI_RIGHT_PIN);

    // Configure Timer2 for PWM motor control.
    TCCR2A = 0b10100011;
    TCCR2B = 0b00000011;

    // Configure Timer0 for PWM motor control.
    TCCR0A = 0b10100011;
    TCCR0B = 0b00000011;

    // Enable pin change interrupts for Port B.
    PCICR |= 0b00000001;

    // Enable pin change interrupt specifically for PB0,
    // which is connected to the color sensor signal.
    PCMSK0 |= 0b00000001;

    // Configure Timer1 in normal counting mode.
    // Timer1 is used to measure the color sensor pulse width.
    TCCR1A = 0;
    TCCR1B = 0b00000001;
```

```
// Start Timer1 at 0.
TCNT1 = 0;

// Enable global interrupts.
sei();

// Make sure the robot begins stopped.
stopMotors();
}

// COLOR SENSOR FUNCTIONS

int getColor()
{
    // Clear previous timer values
    timer = 0;
    TCNT1 = 0;

    // Enable PB0 pin change interrupt so the color sensor pulse can be
    measured.
    PCMSK0 |= 0b00000001;

    // Wait long enough to capture a color sensor pulse
    _delay_ms(20);

    // Disable PB0 pin change interrupt after measurement.
    PCMSK0 &= 0b11111110;

    // Convert measured pulse width into a period value
    period = 2 * timer;

    // Return color sensor reading
    return period;
}
```

```
int getColorStart()
{
    // Reset sum before averaging readings.
    sum = 0;

    // Take 10 readings to reduce noise from the color sensor.
    for (int i = 0; i < 10; i++)
    {
        int per = getColor();
        sum += per;
        _delay_ms(50);
    }

    // Average the 10 readings to define the baseline color.
    int colorS = sum / 10;

    return colorS;
}

int isBlue(int val)
{
    // Returns true if the measured color value falls within the blue range.
    // This function was kept from earlier testing but is not used in the final
    main loop.
    return (val > BLUE_MIN && val < BLUE_MAX);
}

// QTI SENSOR FUNCTIONS

int readLeftQTI(void)
{
    // Read the left QTI sensor.
    // Returns 1 if the sensor is triggered and 0 otherwise
```

```
    return (PINB & (1 << QTI_LEFT_PIN)) != 0;
}

int readRightQTI(void)
{
    // Read the right QTI sensor.
    // Returns 1 if the sensor is triggered and 0 otherwise
    return (PINB & (1 << QTI_RIGHT_PIN)) != 0;
}

// MOVEMENT FUNCTIONS

void forward(void)
{
    // Drive both motors forward.
    // Different PWM values were used because the motors did not behave
    identically.
    OCR0A = 230;
    OCR2A = 200;
    OCR0B = 0;
    OCR2B = 0;
}

void backward(void)
{
    // Drive both motors backward.
    OCR0B = 210;
    OCR2B = 190;
    OCR0A = 0;
    OCR2A = 0;
}
```

```
void turnRight(void)
{
    // Record that the most recent turn direction was right.
    lastTurn = 1;

    // Set motor outputs to rotate the robot to the right
    OCR0A = 140;
    OCR2B = 0;
    OCR0B = 210;
    OCR2A = 0;
}

void turnLeft(void)
{
    // Record that the most recent turn direction was left
    lastTurn = -1;

    // Set motor outputs to rotate the robot to the left
    OCR0B = 0;
    OCR2A = 180;
    OCR0A = 0;
    OCR2B = 120;
}

void stopMotors(void)
{
    // Set all motor PWM outputs to 0 so both motors stop
    OCR0A = 0;
    OCR2A = 0;
    OCR0B = 0;
    OCR2B = 0;

    // Short delay so that robot fully stops before next movement
    _delay_ms(50);
}
```
